Introduction to Internet of Things

Perception Layer - Part 3

RTOS fundamentals with ESP32

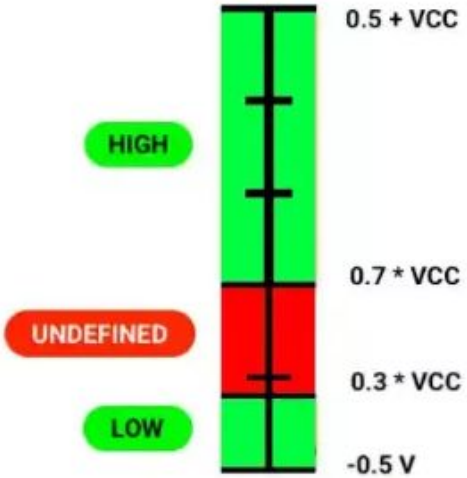
https://www.freertos.org

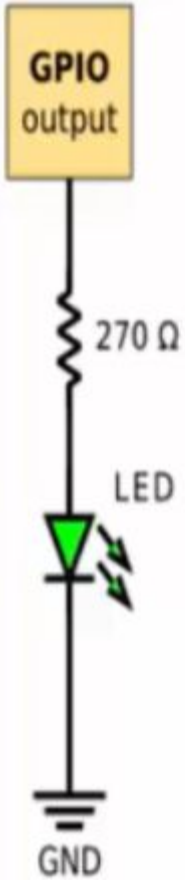
Content

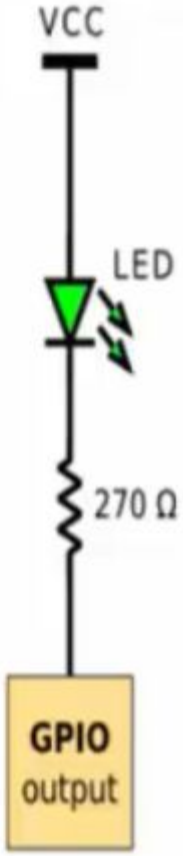
1. GPIOs
2. Interrupts
3. ADC
4. Calibration
5. Practice

1. GPIOs

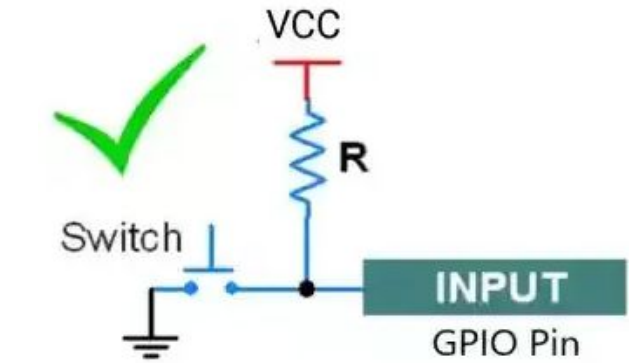
General Purpose Input Outputs

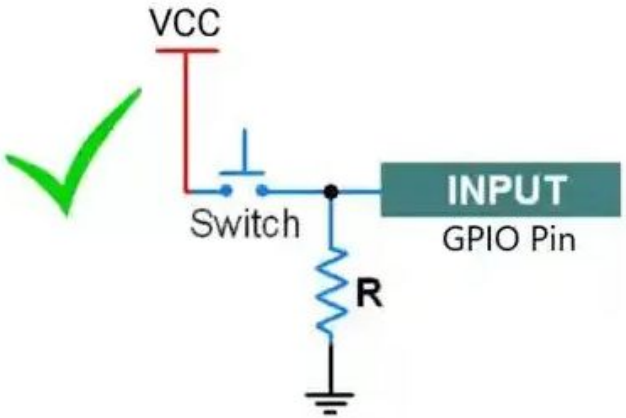


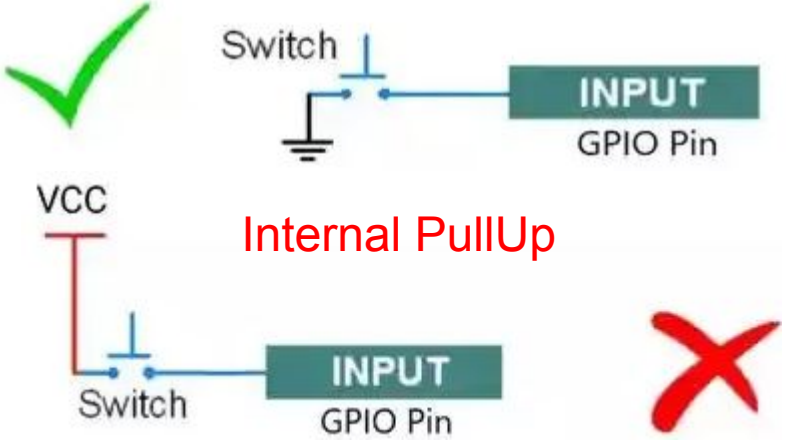


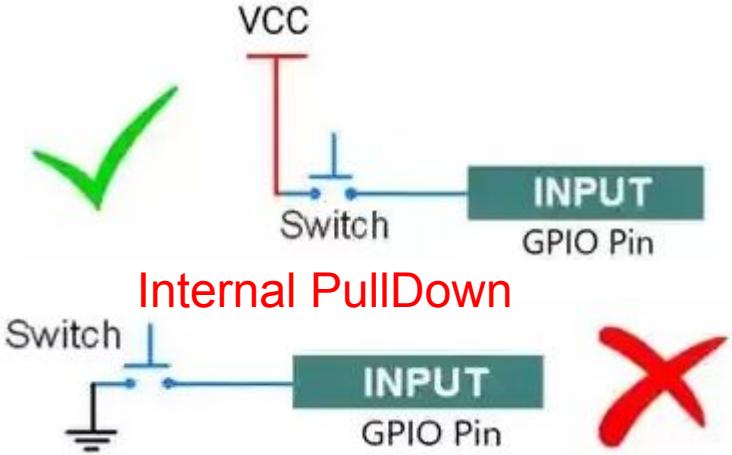


1. GPIOs


VCC
R
Switch
INPUT
GPIO Pin


VCC
Switch
INPUT
GPIO Pin
R


Switch
INPUT
GPIO Pin
VCC
Internal PullUp
Switch
INPUT
GPIO Pin


VCC
Switch
INPUT
GPIO Pin
Internal PullDown
Switch
INPUT
GPIO Pin

1. GPIOs

GPIO configuration

File header

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "driver/gpio.h"
#include "esp_log.h"

#define TAG "LED_BUTTON"

#define LED_GPIO     2
#define BUTTON_GPIO  4

QueueHandle_t buttonQueue;
```

1. GPIOs

GPIO configuration

```c
void gpio_init(void)
{
    // LED como salida
    gpio_config_t led_conf = {
        .pin_bit_mask = (1ULL << LED_GPIO),
        .mode = GPIO_MODE_OUTPUT,
        .pull_up_en = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };
    gpio_config(&led_conf);

    // Botón como entrada con pull-up interno
    gpio_config_t button_conf = {
        .pin_bit_mask = (1ULL << BUTTON_GPIO),
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };
    gpio_config(&button_conf);
}
```

1. GPIOs

Task of button

```c
void taskButton(void *pvParameters)
{
    int lastState = 1;  // Pull-up → 1 = no presionado
    int currentState;

    while (1)
    {
        currentState = gpio_get_level(BUTTON_GPIO);

        // Detectar flanco de bajada
        if (lastState == 1 && currentState == 0)
        {
            ESP_LOGI(TAG, "Botón presionado");
            xQueueSend(buttonQueue, &currentState, portMAX_DELAY);

            vTaskDelay(pdMS_TO_TICKS(200)); // debounce simple
        }

        lastState = currentState;
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}
```

1. GPIOs

Task led

```
void taskButton(void *pvParameters)
{
    int lastState = 1;  // Pull-up → 1 = no presionado
    int currentState;

    while (1)
    {
        currentState = gpio_get_level(BUTTON_GPIO);

        // Detectar flanco de bajada
        if (lastState == 1 && currentState == 0)
        {
            ESP_LOGI(TAG, "Botón presionado");
            xQueueSend(buttonQueue, &currentState, portMAX_DELAY);

            vTaskDelay(pdMS_TO_TICKS(200)); // debounce simple
        }

        lastState = currentState;
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}
```

1. GPIOs

main function

```
void app_main(void)
{
    gpio_init();

    buttonQueue = xQueueCreate(5, sizeof(int));

    xTaskCreate(taskButton, "TaskButton", 2048, NULL, 2, NULL);
    xTaskCreate(taskLed,    "TaskLed",    2048, NULL, 2, NULL);
}
```

2. Interrupts

Interrupt version

```c
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "driver/gpio.h"
#include "esp_log.h"

#define TAG "LED_ISR"

#define LED_GPIO     2
#define BUTTON_GPIO  4

QueueHandle_t buttonQueue;
```

2. Interrupts

Interrupt Sub-Routine

```
static void IRAM_ATTR button_isr_handler(void* arg)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    int event = 1;

    xQueueSendFromISR(buttonQueue, &event, &xHigherPriorityTaskWoken);

    if (xHigherPriorityTaskWoken)
    {
        portYIELD_FROM_ISR();
    }
}
```

2. Interrupts

Init GPIOs

```c
void gpio_init(void)
{
    // LED salida
    gpio_config_t led_conf = {
        .pin_bit_mask = (1ULL << LED_GPIO),
        .mode = GPIO_MODE_OUTPUT,
        .pull_up_en = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };
    gpio_config(&led_conf);

    // Botón entrada con pull-up e interrupción
    gpio_config_t button_conf = {
        .pin_bit_mask = (1ULL << BUTTON_GPIO),
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_NEGEDGE  // flanco descendente
    };
    gpio_config(&button_conf);

    gpio_install_isr_service(0);
    gpio_isr_handler_add(BUTTON_GPIO, button_isr_handler, NULL);
}
```

2. Interrupts

LED task

```c
void taskLed(void *pvParameters)
{
    int received;
    int ledState = 0;

    while (1)
    {
        if (xQueueReceive(buttonQueue, &received, portMAX_DELAY))
        {
            ledState = !ledState;
            gpio_set_level(LED_GPIO, ledState);

            ESP_LOGI(TAG, "LED cambiado a: %d", ledState);

            // Debounce simple por software
            vTaskDelay(pdMS_TO_TICKS(200));
        }
    }
}
```

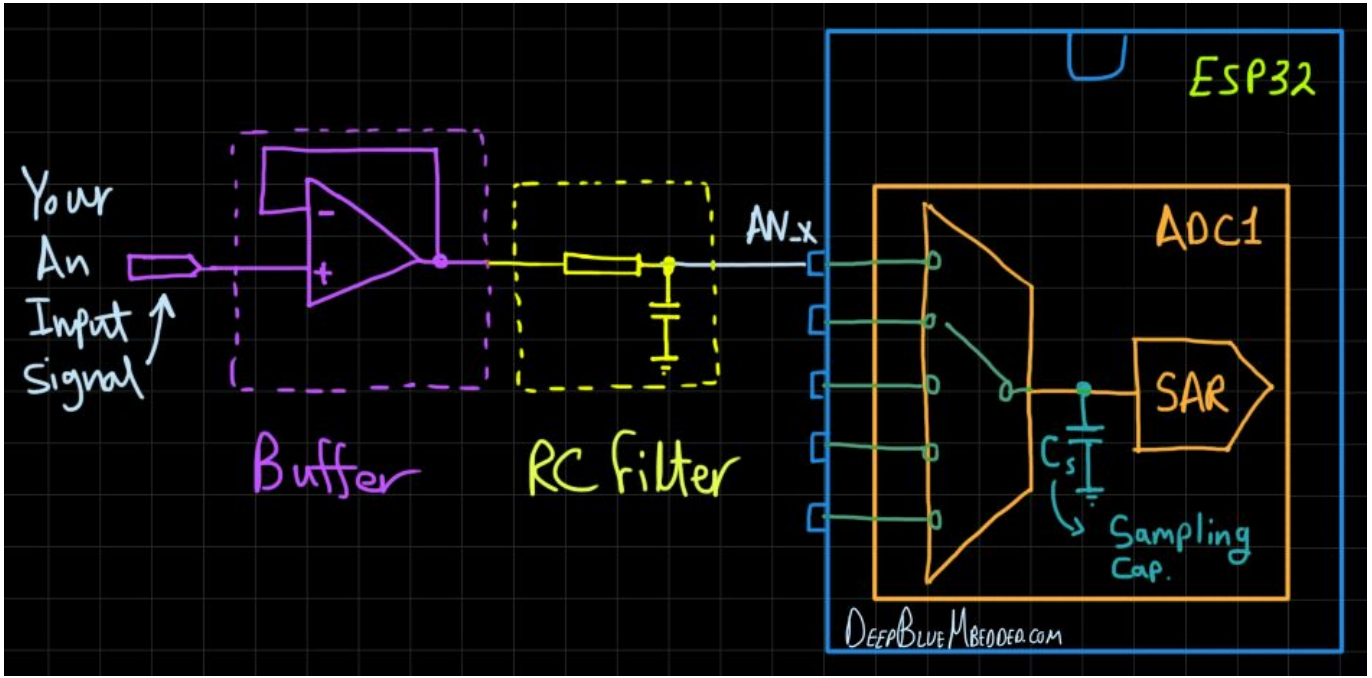
2. Interrupts

main function

```c
void app_main(void)
{
    gpio_init();

    buttonQueue = xQueueCreate(10, sizeof(int));

    if (buttonQueue == NULL)
    {
        ESP_LOGE(TAG, "Error creando la cola");
        return;
    }

    xTaskCreate(taskLed, "TaskLed", 2048, NULL, 2, NULL);
}
```

3. ADC

$$\textbf{ADC Resolution} = \frac{Vref}{(2^n - 1)}$$



3. ADC

Header part

Attenuation

0 dB: Full-scale voltage Vmax = 1.1V

2.5 dB: Full-scale voltage Vmax = 1.5V

6 dB: Full-scale voltage Vmax = 2.2V

11 dB: Full-scale voltage Vmax=3.3V

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"

#include "driver/adc.h"
#include "esp_log.h"

#define TAG "ADC_RTOS"

#define ADC_CHANNEL ADC1_CHANNEL_6   // GPIO34
#define ADC_WIDTH ADC_WIDTH_BIT_12
#define ADC_ATTEN ADC_ATTEN_DB_11

#define RTOS_delay(x) vTaskDelay(pdMS_TO_TICKS(x))
```

3. ADC

ADC configuration

ADC task

```c
QueueHandle_t adcQueue;

void adc_init()
{
    adc1_config_width(ADC_WIDTH);
    adc1_config_channel_atten(ADC_CHANNEL, ADC_ATTEN);
}

void taskADC(void *pvParameters)
{
    int adc_reading;

    while (1)
    {
        adc_reading = adc1_get_raw(ADC_CHANNEL);

        xQueueSend(adcQueue, &adc_reading, portMAX_DELAY);

        RTOS_delay(500);
    }
}
```

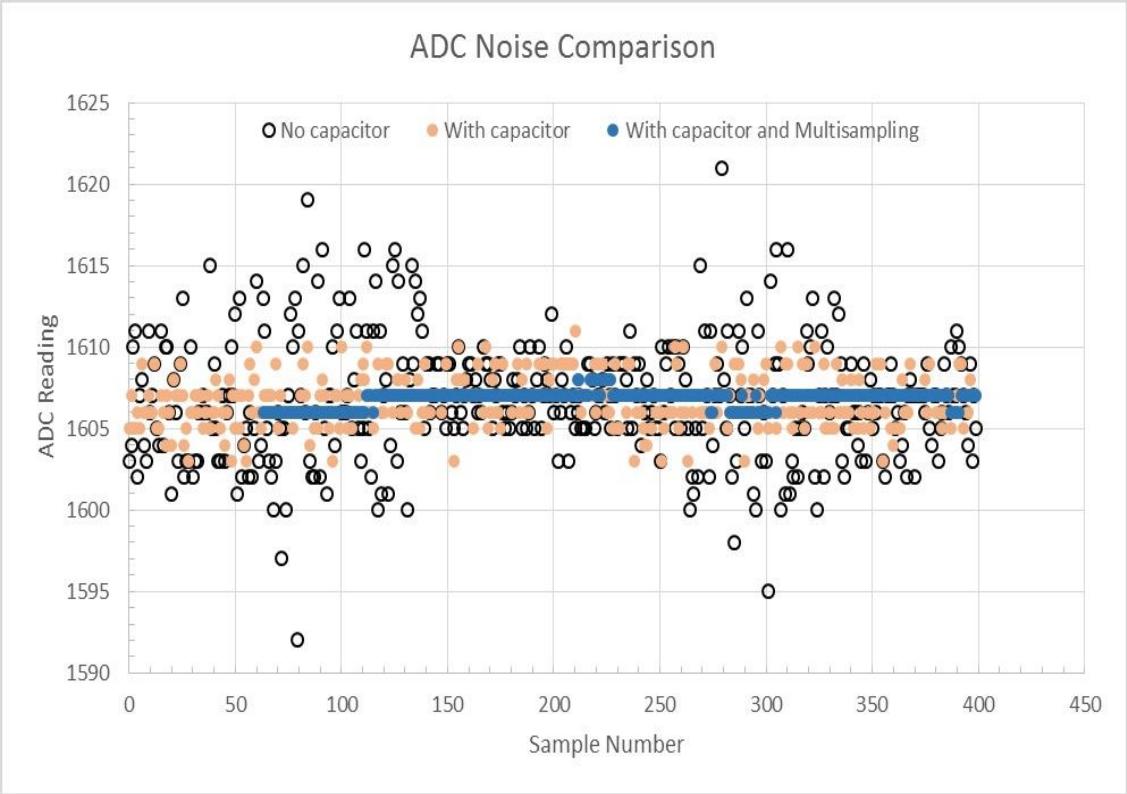
3. ADC

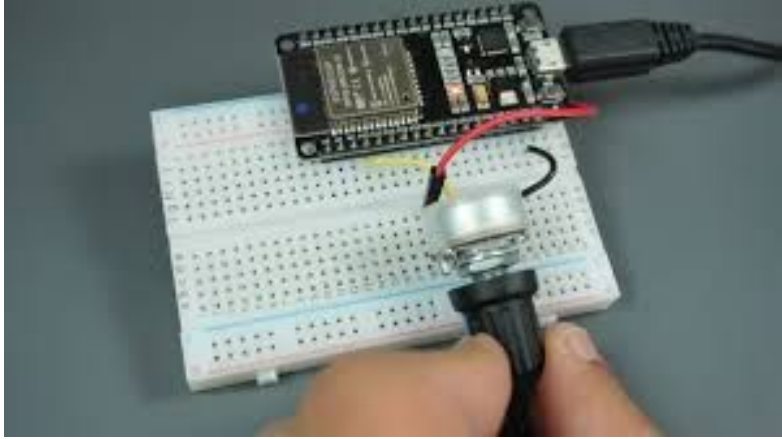
Task for get data

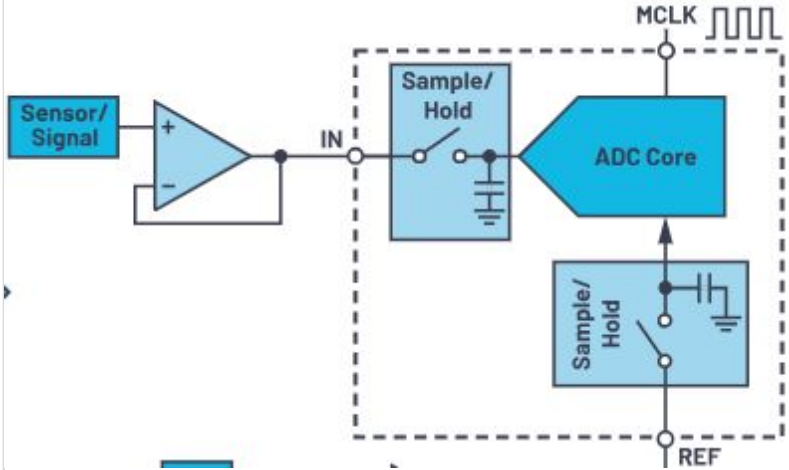
main function

```c
void taskProcessing(void *pvParameters)
{
    int received_value;

    while (1)
    {
        if (xQueueReceive(adcQueue, &received_value, portMAX_DELAY))
        {
            ESP_LOGI(TAG, "ADC value: %d", received_value);
        }
    }
}

void app_main(void)
{
    adc_init();

    adcQueue = xQueueCreate(10, sizeof(int));

    xTaskCreate(taskADC, "TaskADC", 2048, NULL, 2, NULL);
    xTaskCreate(taskProcessing, "TaskProcessing", 2048, NULL, 1, NULL);
}
```

4. Calibration







5. Practice

Implementation of a calibration example

Implementation of a complete Data acquisition application in FreeRTOS:

- Read raw data
- Use the calibration to improve raw data
- Use DMA to avoid CPU intervention
- Consume the data from DMA
- Filter the data using moving average